# Setting the Administrator Password

How to set, change or remove the password that signs in as the administrator of ImageSkinForLLM, on a laptop (Windows 10, Windows 11 or Mac) and on the hosted site on Oracle Cloud, and how to let only the laptops you list sign in.

**The short version**

1. The password is set in a file, `admin.toml`, on the computer that runs the app. There is no sign-up and no way to set or change it in the app: only someone who can sign in to that computer can.
2. Copy the template `admin.example.toml`, from the top of the project folder, to where the app looks for `admin.toml`, and put the password in it.
3. Save the file. The app reads it again by itself: **no restart is needed**, and the new password works at once.
4. Sign in on the app's **Settings** page, under **Administrator**.
5. Optional: to let only your own laptops sign in, list their IDs in a second file, `admin-laptops.toml`, beside `admin.toml` (see Only the laptops you list).

Time needed: about 5 minutes.

> **Never commit `admin.toml`.** It holds the password. The project's `.gitignore` keeps it out of git; only the empty template, `admin.example.toml`, is in the repository.

## Where the file goes

| Where the app runs | The file |
|---|---|
| Windows 10 or 11 | `C:\Users\*you*\.imageskin\admin.toml` (`%USERPROFILE%\.imageskin\admin.toml`) |
| Mac | `~/.imageskin/admin.toml` (`/Users/*you*/.imageskin/admin.toml`) |
| The hosted site (Docker on Oracle Cloud) | `~/ImageSkinForLLM/deploy/admin/admin.toml` on the machine |

The `.imageskin` folder is the app's home folder, made the first time the app runs. If `IMAGESKIN_HOME` is set, the file goes in that folder instead. A config file can also name another place with `admin_file = "..."` (see `config.example.toml`); the hosted site's `deploy/config.toml` does this, pointing at the `deploy/admin` folder.

# What to write in the file

The file has two lines. Fill in one of them.

```
password_hash = ""
password = ""
```

### Best: a hash of the password

A hash lets the app check the password without the password itself being stored on the disk. Make it with the app's own command, typing the password twice (nothing shows as you type):

```
imageskin admin-password-hash
```

It prints a line such as

```
password_hash = "scrypt$16384$8$1$Qm9vbXNoYWth...$9yGd7z...="
```

Paste that whole line into `admin.toml` in place of its `password_hash = ""` line, and save. Run the command in the project's Python environment, the same way you run `imageskin serve`; on the hosted site, run it in the app's container (below).

### Or: the password as it is

```
password = "a long passphrase you will remember"
```

This is simpler, but anyone who can read the file can read the password, so keep the file readable only by you (below). If both lines are filled in, the hash is used.

> **A password with a backslash (\) or a double quote (")** needs care in this file format. Use single quotes instead, `password = 'my "quoted" pass\word'`, or use the hash, which never has them.

# On a Windows 10 or 11 laptop

1. Open **Command Prompt** and go to the project folder (where `admin.example.toml` is), for example:

   ```
   cd %USERPROFILE%\github\ImageSkinForLLM
   ```

2. Copy the template into the app's home folder:

   ```
   mkdir "%USERPROFILE%\.imageskin"
   copy admin.example.toml "%USERPROFILE%\.imageskin\admin.toml"
   ```

   (`mkdir` says the folder already exists if the app has run before; that's fine.)

3. Make the hash, in the project's Python environment:

```
.venv\Scripts\activate
imageskin admin-password-hash
```

4. Open the file in Notepad, paste the `password_hash = "..."` line over the empty one, and save:

```
notepad "%USERPROFILE%\.imageskin\admin.toml"
```

5. Make the file readable only by you:

```
icacls "%USERPROFILE%\.imageskin\admin.toml" /inheritance:r /grant:r "%USERNAME%:F"
```

In PowerShell, the same steps are `New-Item -ItemType Directory -Force "$env:USERPROFILE\.imageskin"`, `Copy-Item admin.example.toml "$env:USERPROFILE\.imageskin\admin.toml"`, `.venv\Scripts\Activate.ps1`, `notepad "$env:USERPROFILE\.imageskin\admin.toml"` and `icacls "$env:USERPROFILE\.imageskin\admin.toml" /inheritance:r /grant:r "${env:USERNAME}:F"`.

## On a Mac

1. Open **Terminal** and go to the project folder, for example:

```
cd ~/github/ImageSkinForLLM
```

2. Copy the template into the app's home folder and make it readable only by you:

```
mkdir -p ~/.imageskin
cp admin.example.toml ~/.imageskin/admin.toml
chmod 600 ~/.imageskin/admin.toml
```

3. Make the hash, in the project's Python environment:

```
source .venv/bin/activate
imageskin admin-password-hash
```

4. Open the file, paste the `password_hash = "..."` line over the empty one, and save (in nano, Ctrl+O, Enter, then Ctrl+X):

```
nano ~/.imageskin/admin.toml
```

# On the hosted site (Oracle Cloud)

> **Two different passwords.** The hosted site already asks for a name and password before anything is shown (Caddy's, in `deploy/.env`; see `deploy/README.md`). That one lets anyone you give it to use the site. The administrator password here is a second one, inside the app, for the admin view only.

1. Connect to the machine from your laptop, as in step 4 of `deploy/README.md`, and go to the deploy folder:

   ```
   ssh -i ~/.ssh/oracle.key ubuntu@your-machine's-address
   cd ~/ImageSkinForLLM/deploy
   ```

2. **Once only**, after updating to the version with the admin view: get it and start the app again, so its container gets the `deploy/admin` folder:

   ```
   git pull
   docker compose up -d --build
   ```

   This is the only restart: later password changes need none.

3. Copy the template and make it readable only by you:

   ```
   cp ../admin.example.toml admin/admin.toml
   chmod 600 admin/admin.toml
   ```

   The app in its container runs as the same user as you on the machine (user 1000, `ubuntu`), so it can still read the file.

4. Make the hash in the app's container:

   ```
   docker compose exec app imageskin admin-password-hash
   ```

5. Open the file, paste the line over the empty one, and save (Ctrl+O, Enter, Ctrl+X):

   ```
   nano admin/admin.toml
   ```

6. Check the app found it: its log says `Admin password file read` with `"password_set": true` once the Settings page has been opened, or the Settings page offers the password field instead of saying signing in is off:

   ```
   docker compose logs app | grep "Admin password file read"
   ```

# Signing in and out

1. Open the app's **Settings** page (in either view) and scroll to **Administrator**.

2. Type the password and click **Sign in as administrator**. An **Admin** link appears in the top bar; it opens the admin view.
3. To sign out, click **Sign out as administrator** on the Settings page. Without a click or key press on the Settings page or in the admin view for 30 minutes (an admin setting), you are signed out by yourself, and the page says so.

After 5 failed sign-ins in a row, the next try waits a minute.

## Changing, removing or forgetting the password

- **To change it:** make a new hash (or type the new password) and save the file over the old line. It works at once, with no restart. Anyone signed in with the old password is signed out, and the page says the password was changed.
- **To turn signing in off:** empty both lines (`password_hash = ""` and `password = ""`) or delete the file. Anyone signed in is signed out, and the Settings page says signing in as the administrator is off.
- **If it is forgotten:** there is no way to recover it, as the file holds only its hash. Whoever can sign in to the computer sets a new one, as above.

# Only the laptops you list

With just a password, anyone who learns it can sign in from any computer. A second file, `admin-laptops.toml`, beside `admin.toml`, lists the laptops allowed to sign in. Once it exists, a laptop not in it can't sign in even with the right password, and its **Settings** page shows nothing of the administrator at all: no **Administrator** section, and no **Admin** link. With no such file, any laptop can sign in, as before.

| Where the app runs | The file |
|---|---|
| Windows 10 or 11 | `%USERPROFILE%\.imageskin\admin-laptops.toml` |
| Mac | `~/.imageskin/admin-laptops.toml` |
| The hosted site | `~/ImageSkinForLLM/deploy/admin/admin-laptops.toml` on the machine (in `deploy/admin`, not `deploy`) |

As with the password, the app reads the file again whenever it is saved: **no restart is needed**. It is never committed; only the template, `admin-laptops.example.toml`, is in the repository.

## 1. Copy this laptop's ID

1. On the laptop to allow, open the app's **Settings** page in the browser you will sign in with: the hosted site's `https://` address, or `http://127.0.0.1:8000` for the app on that laptop.
2. Find **This laptop**. Before there is a list, or once this laptop is in it, it is at the end of the **Administrator** section; on a laptop the list leaves out, it is the only thing shown about it. It reads `This laptop's ID: LT-3F9K-Q7MZ-AB2C-D4EF-GH5J`.
3. Click **Copy**. It copies three lines, ready to paste:

   ```
   [[laptop]]
   name = "My laptop"
   id = "LT-3F9K-Q7MZ-AB2C-D4EF-GH5J"
   ```

   Send them to the computer the file is on, for example by email to yourself. The ID is not a secret: it only names the laptop, and can't be used to sign in from anywhere else.

> On a laptop the list leaves out, the ID is also in the browser's console (F12, then **Console**), as `This laptop's ID: LT-…`.

## 2. Make the file and list the laptop

Copy the template once, from the project folder, then paste each laptop's three lines at the end of the file. Change `name` to something you will recognise; only `id` is checked.

### Windows (Command Prompt, in the project folder)

```
copy admin-laptops.example.toml "%USERPROFILE%\.imageskin\admin-laptops.toml"
notepad "%USERPROFILE%\.imageskin\admin-laptops.toml"
```

### Mac (Terminal, in the project folder)

```
cp admin-laptops.example.toml ~/.imageskin/admin-laptops.toml
nano ~/.imageskin/admin-laptops.toml
```

### The hosted site (after connecting with ssh, as above)

```
cd ~/ImageSkinForLLM/deploy
cp ../admin-laptops.example.toml admin/admin-laptops.toml
nano admin/admin-laptops.toml
```

Two laptops look like this; the lines starting with `#` from the template can stay:

```
[[laptop]]
name = "Larry's laptop"
id = "LT-3F9K-Q7MZ-AB2C-D4EF-GH5J"

[[laptop]]
name = "Office desktop, Firefox"
id = "LT-M2QX-7HDK-W9ZP-4RTA-C6NB"
```

Save, then reload the **Settings** page on the laptop: the **Administrator** section and its password field are back. On the hosted site, the log says how many laptops it found: `docker compose logs app | grep "Admin laptops file read"`.

> **Make the file only once your own laptop's ID is in it.** A file that lists no laptop, or that isn't valid, lets **no** laptop sign in. If that happens, fix or delete the file on the computer: no restart is needed.

## Another browser, another laptop, or taking one out

- **Each browser has its own ID.** Chrome and Firefox on the same laptop, or two Chrome profiles, are two entries. Copy the ID from each browser you want to use, as in step 1.
- **Another laptop:** copy its ID from its own **Settings** page and add its three lines to the file.
- **Taking one out:** delete its three lines and save. If it is signed in, it is signed out at once, and its page says only "Signed out as administrator."
- **Letting any laptop in again:** delete `admin-laptops.toml` (or rename it).

## After clearing the browser's data

The ID belongs to a key the browser keeps for the site. Clearing the browser's data for the site (cookies and site data), resetting the browser, or a new browser profile makes a new key, so a new ID. The laptop then can't sign in: open **Settings**, copy the new ID, put it in the file in place of

the old one, and save. A private or incognito window gets a new ID each time it is opened, so it can't be listed.

### Only on HTTPS or localhost

Browsers make the key only on a secure address: the hosted site's `https://` address, or `localhost` (and `127.0.0.1`) on the laptop running the app. At a plain `http://` address, such as another computer's address on your home network, **This laptop** says "This laptop's ID can't be shown at this address." Once there is a list, no laptop can sign in from such an address. Without a list, signing in there works as before.

## If it doesn't work

- **The Settings page says signing in as the administrator is off:** the app found no password. The file is in another folder, has another name (Windows may have saved it as `admin.toml.txt`: in Notepad, choose **All files** when saving), both lines are empty, or the file isn't valid. The log line `Admin password file read` shows the path the app reads and `password_set`; `Could not read the admin password file` gives the error.
- **"Admin login failed with error code A-..."** Every failed sign-in says only this, so that someone guessing learns nothing. The log line `Admin sign-in refused` with that code gives the reason, never the password: find it with `docker compose logs app | grep A-7K3F9Q` on the hosted site, using the code the page showed. The reason is `wrong password` (check for a space at the end, or a hash pasted only in part: it must be one line, in double quotes), `no password set`, `waiting` after 5 failures in a row (wait a minute; the count starts again after a sign-in that works), or one of the laptop reasons below. On a laptop, the log is the terminal running `imageskin serve`. The FAQ's "Admin login failed with an error code. How do I find out why?" has every reason and the commands for each computer.
- **The Settings page shows only This laptop:** there is an `admin-laptops.toml` and this browser's ID isn't in it. Check the ID in the file is the one the page shows now (it changes after clearing the browser's data), that the file is in `deploy/admin` on the hosted machine, not `deploy`, and that its log line `Admin laptops file read` counts the laptops you expect. `Could not read the admin laptops file` gives the error in a file that isn't valid.
- **The laptop reasons in the log:** `laptop not listed` (the browser's ID isn't in the file); `no laptop proof` (the page was opened at a plain `http://` address, or the browser couldn't make its key); `challenge expired` (the sign-in took more than a minute after the page asked for it: try again); `bad laptop proof` (the browser sent a proof that doesn't check out: reload the page and try again). Each line also says `password_ok`, whether the password was right.
- **On the hosted site, a change isn't seen:** check you edited `deploy/admin/admin.toml` on the machine, and that `docker compose exec app cat /etc/imageskin/admin/admin.toml` shows the new line. If it says no such file, the container was started before step 2: run `docker compose up -d`.